Reliability, Consistency, and Threading Architectures in Cloud Message Brokers: A Distributed-Systems Performance Study

Bhuvana Komati[1], Himanshu Jain[2], and Vidhyacharan Bhaskar[3]*

[123]Department of Computer Engineering

San Jose State University

One Washington Square, San Jose, CA 95192, USA

Email: bhuvana.komati@sjsu.edu, himanshu.jain@sjsu.edu, charan.bhaskar@sjsu.edu

Corresponding author email: charan.bhaskar@sjsu.edu

October 2026

Abstract

Cloud message brokers such as AWS SQS, RabbitMQ, and Apache Kafka must do far more than move messages quickly: they must guarantee at-least-once delivery, durability, ordering, and—for transactional workloads—atomicity and consistency across replicas. These reliability and consistency mechanisms, together with the thread-pool architecture that executes them, impose a performance cost that classical queueing models ignore. This paper is a companion to our earlier study of heavy-tailed service-time variability; here we hold the service distribution fixed and instead quantify the performance impact of the broker's distributed-systems machinery. Using a SimPy discrete-event framework we model and measure (i) reliability mechanisms—visibility timeout, replication, and message ordering; (ii) consistency protocols—Two-Phase Commit (2PC), Raft consensus, Vector Clocks, and the CAP trade-off; (iii) threading architectures—dedicated versus shared thread pools; and (iv) queueing extensions—heterogeneous servers, finite capacity, priority, and Erlang-k service. Our central findings are quantitative. 2PC adds roughly a 36% service-time overhead under a three-replica, 10 ms round-trip configuration, lowering effective service rate by about 26%. A shared thread pool is 10–15% slower than a dedicated pool at moderate load but degrades catastrophically near saturation (4.4× the baseline response time at $\rho = 0.9$), while a dedicated pool caps admissible throughput at N/2 connections. Server heterogeneity is dominated by service-rate variance rather than aggregate capacity: a configuration with 1.7% more capacity but higher variance was orders of magnitude slower, and a shortest-queue dispatch policy recovered almost all of the loss. Replication factor three raises reliability and throughput at the cost of 3× storage, and longer visibility timeouts reduce redelivery retries but inflate tail latency. We report measured results where the simulator persists them and clearly identify the protocols for which only structural/qualitative validation is available.

Keywords: Two-Phase Commit, Raft Consensus, CAP Theorem, Vector Clocks, Thread Pool Architecture, Visibility Timeout, Replication, Message Ordering, Cloud Message Broker, Discrete-Event Simulation

1 Introduction

Cloud message brokers are the connective tissue of modern distributed systems, decoupling producers from consumers and absorbing load spikes through asynchronous queues. Production systems such as AWS Simple Queue Service (SQS), RabbitMQ, and Apache Kafka are not, however, simple FIFO buffers. To deliver the guarantees that applications depend on—no message loss, survivable storage, ordered or causally consistent delivery, and in some cases transactional atomicity—they layer a substantial amount of distributed-systems machinery on top of the raw queue: visibility timeouts and redelivery, multi-replica storage with quorum acknowledgement, consensus for leader election and

log replication, and atomic-commit protocols. Each of these mechanisms consumes time and threads, and that cost is invisible to a textbook M/M/N analysis.

In a companion paper we studied one axis of broker performance—the effect of heavy-tailed (Pareto) service-time distributions on waiting time and tail latency, the resulting “reality gap” between M/M/N predictions and observed P99 latency, and M/G/N approximations and Extreme Value Theory as remedies. That work deliberately held the broker's reliability and consistency features fixed. The present paper is the orthogonal study: we hold the service distribution fixed and quantify the performance consequences of the reliability, consistency, and concurrency mechanisms themselves. Together the two papers give a complete picture of why real brokers behave differently from classical queueing predictions —one paper attributing the gap to workload, the other to protocol.

The objectives of this paper are fourfold. First, we model three reliability mechanisms—visibility timeout, replication, and message ordering—and measure the trade-offs they impose between durability, retry rate, tail latency, and storage. Second, we characterize four consistency and coordination protocols—Two-Phase Commit, Raft consensus, Vector Clocks, and the CAP trade-off—and quantify, where our simulator records it, the latency and throughput tax of strong consistency. Third, we compare dedicated and shared thread-pool architectures and derive the operating regime in which each is preferable. Fourth, we extend the baseline queue to heterogeneous servers, finite capacity, priority classes, and Erlang-k service, isolating the effect of service-rate variance and dispatch policy. Throughout, we distinguish results the simulator persists as numeric output from those for which only structural or qualitative validation exists.

1.1 Background and Related Work

The theoretical foundations of distributed coordination predate cloud computing. Lamport's logical clocks [2] established the happened-before relation, later refined into vector clocks by Fidge [3] and Mattern [4] to capture causality precisely and detect concurrency. Atomic commitment across participants was formalized by Gray [7] as the Two-Phase Commit protocol, whose blocking behavior under coordinator failure was analyzed by Gray and Lamport [8]. Consensus—agreement despite failures—was shown by Fischer, Lynch, and Paterson [11] to be impossible in a purely asynchronous system with even one crash, motivating partially-synchronous, leader-based protocols: Paxos [6] and the more operationally tractable Raft algorithm of Ongaro and Ousterhout [5].

The tension these protocols navigate was crystallized by Brewer's CAP conjecture [9] and proved by Gilbert and Lynch [10]: under a network partition a system must sacrifice either consistency or availability. Vogels [12] articulated the eventual-consistency model that production stores adopt to favor availability, and DeCandia et al.'s Dynamo [13] is the canonical highly-available, eventually-consistent design that influenced modern brokers and key-value stores. Dean and Barroso [14] documented how, at scale, tail latency rather than mean latency governs user experience, which motivates our emphasis on P99 as well as mean metrics.

On the queueing side, Erlang's blocking and delay formulas [16], Kleinrock's foundational treatment [15], Little's Law [17], and Harchol-Balter's modern synthesis [18] supply the analytical machinery we use for finite-capacity, priority, and heterogeneous-server models. Our base system is the tandem cloud-broker model of Li et al. [1], whose visibility-timeout and message-ordering semantics (their Sections 3 and 6.4) we implement directly, and whose reliability parameterization motivates our replication study. Where Li et al. use Colored Petri Nets, we use discrete-event simulation in SimPy [20] and complement it with closed-form queueing results [21] where they exist.

2 System Model and Simulation Framework

All results in this paper are produced by a single discrete-event simulation framework implemented in Python using SimPy 4.1.1 [20]. Messages arrive as a Poisson process of rate λ; service is executed by a pool of N threads modeled as SimPy resources; and reliability or consistency mechanisms are inserted either as additional service-time components (2PC overhead, replication acknowledgement) or as explicit protocol state machines (Raft nodes, vector-clock delivery queues, visibility-timeout managers). Every experiment discards an initial warm-up interval to remove transient bias and fixes a random seed for reproducibility; configurations are validated for stability ($\rho < 1$) through type-safe Pydantic models. The default broker parameters follow Li et al. [1]: a one-way network link delay D of 10 ms, failure probability p in the range 0.01–0.2, and service rate $\mu = 12$ messages/second/thread unless stated otherwise. The common system parameters are summarized in Table 1.

Table 1: Common system and reliability parameters used across experiments.

Parameter	Symbol	Default value
Service rate per thread	μ	12 msg/s
Network one-way link delay	D	10 ms
2PC round-trip time (RTT)	RTT	10 ms (σ = 2 ms)
Replica availability	—	0.99
2PC vote timeout	T_timeout	1.0 s
Default replicas / replication factor	n, R	3
Visibility timeout (SQS default)	VT	30 s
Max receive count (poison threshold)	—	3
Thread-pool overhead coefficient	α	0.1
Threads per dedicated connection	—	2

The remainder of the paper is organized by mechanism. Section 3 treats reliability (visibility timeout, replication, ordering); Section 4 treats consistency and consensus (2PC, Raft, Vector Clocks, CAP); Section 5 compares threading architectures; Section 6 presents queueing extensions (heterogeneous, finite-capacity, priority, Erlang-k); Section 7 consolidates the experimental findings and states limitations; and Section 8 concludes.

3 Reliability Mechanisms

Reliable delivery is achieved not by preventing failures but by detecting and recovering from them. We model the three mechanisms a broker uses for this purpose and measure the performance they trade for reliability.

3.1 Visibility Timeout and Redelivery

When a consumer receives a message, the broker does not delete it; it hides the message for a visibility-timeout interval VT and starts a timer. If the consumer acknowledges within VT, the message is deleted; if the timer expires, the message becomes visible again and is redelivered, with its receive counter incremented. A message whose receive count reaches the poison threshold (default 3) is routed to a dead-letter queue rather than redelivered indefinitely. The mechanism therefore exposes three rates:

$$\text{ack_rate} = \text{total_acks} / \text{total_receives} \quad (1)$$

$$\text{timeout_rate} = \text{total_timeouts} / \text{total_receives} \quad (2)$$

$$\text{dlq_rate} = \text{total_dlq} / \text{total_receives}, \quad \text{invisible_until} = t_{recv} + VT \quad (3)$$

VT is a tuning knob with opposing effects: too short, and slow-but-healthy consumers are falsely timed out and the message is redelivered needlessly; too long, and a genuinely failed delivery sits invisible for a long time before any retry, inflating tail latency. We swept VT ∈ {10, 30, 60, 120} s with λ = 10 msg/s, mean processing time 0.5 s, failure probability 0.1, three storage nodes, replication factor two, over 500 s with a 50 s warm-up. Table 2 reports the measured outcome.

Table 2: Measured effect of visibility timeout. Mean retries fall as VT grows, but P99 latency rises because failed messages wait longer before redelivery. (Source: visibility_timeout_results.csv.)

VT (s)	Mean retries	Mean latency (ms)	P99 latency (ms)	Throughput (msg/s)
10	0.0711	51.5	156.1	16.03
30	0.0565	33.2	134.0	16.29
60	0.0452	44.6	180.2	16.33
120	0.0397	37.0	222.4	16.35

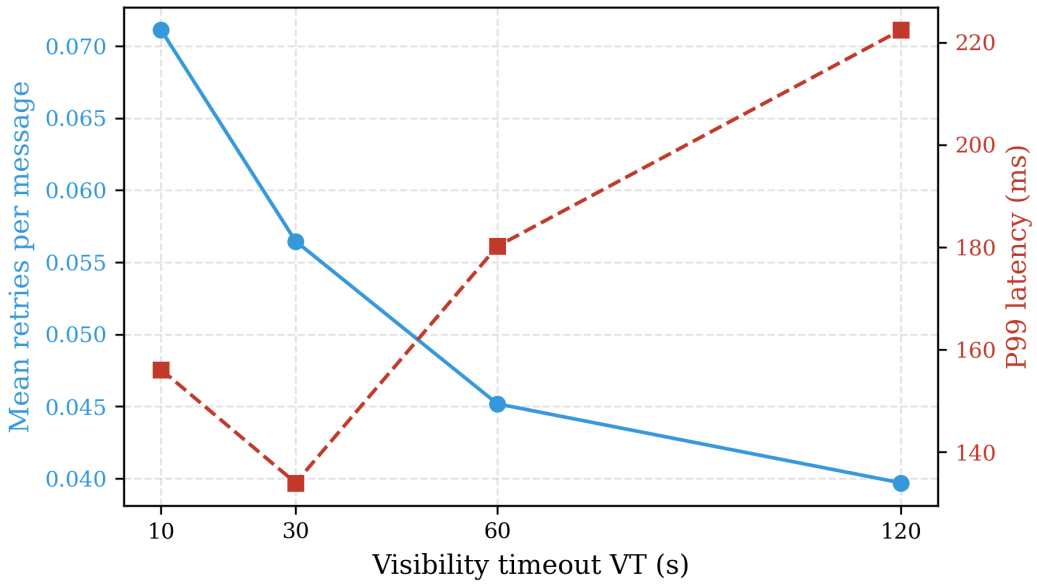


Figure 1: Visibility-timeout trade-off. Increasing VT reduces the mean retry count (left axis) but inflates P99 latency (right axis), because failed messages wait longer before redelivery.

The trend confirms the trade-off analytically expected: increasing VT from 10 s to 120 s cuts the mean retry count by 44% (0.0711 → 0.0397) and modestly raises throughput, but P99 latency grows by 43% (156 → 222 ms) because each failed message incurs a longer invisible wait before its retry. A moderate VT of 30 s—the AWS SQS default—gives the best mean latency in our sweep, supporting the industry default as a reasonable balance point.

3.2 Replication and Quorum Acknowledgement

Durability is obtained by storing each message on more than one node. Our distributed broker places messages using a consistent-hash ring with 150 virtual nodes per physical node; each message is written to a replica set consisting of a primary plus the next R−1 distinct clockwise nodes, where the replication factor R is capped at the number of nodes. A receive is acknowledged successfully when a strict majority of the replica set confirms it:

$$\text{replication_factor} = \min(R, \text{num_nodes}) \tag{4}$$

$$\text{success} \iff \text{success_count} > \lfloor \text{len(replica_set)} / 2 \rfloor \tag{5}$$

$$\text{unique_messages} \approx \text{total_stored} / \text{replication_factor} \tag{6}$$

We swept R ∈ {1, 2, 3} with λ = 100 msg/s, mean processing time 0.05 s, failure probability 0.1, VT = 30 s, three nodes, over 500 s with a 50 s warm-up. Table 3 reports the measured outcome. The "processing ratio" column is the number of processed deliveries divided by the number of distinct published messages; it exceeds unity at R = 3 because replicated copies and retried deliveries are each counted as processing work, so it should be read as a relative processing/throughput ratio rather than a success probability.

Table 3: Measured effect of replication factor. Higher R increases delivered throughput and reliability at the cost of storage amplification (total stored grows while distinct messages per store falls). (Source: replication_factor_results.csv.)

Replication factor R	Processing ratio	Throughput (msg/s)	Total stored	Unique messages
1	0.359	35.8	31,960	31,960
2	0.906	90.8	52,430	26,215
3	1.800	179.8	53,715	17,905

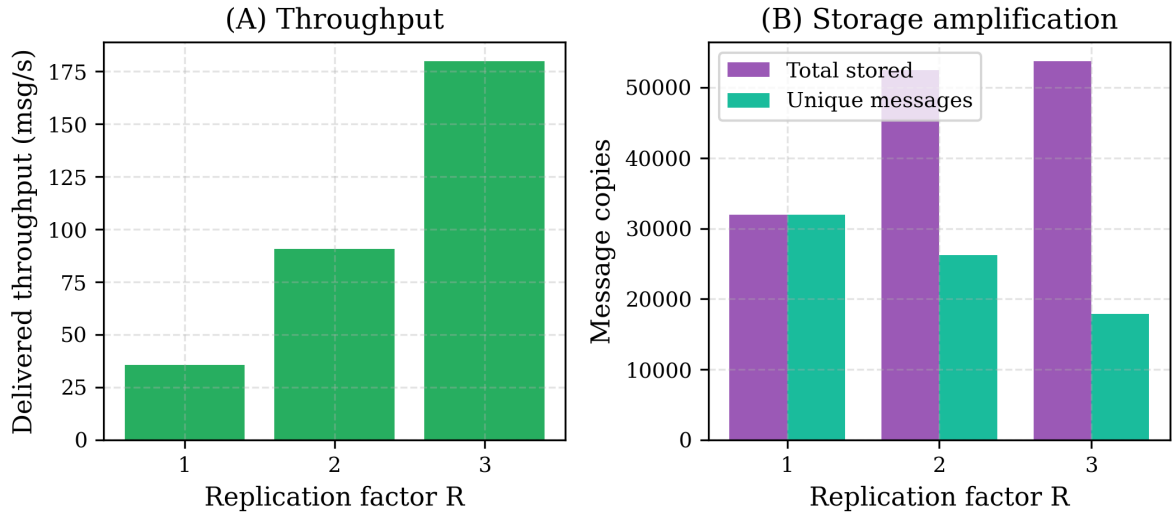


Figure 2: Replication factor. (A) Delivered throughput rises sharply with R; (B) storage cost grows (total stored) while the number of distinct messages per store falls.

Moving from a single copy to three copies raises delivered throughput roughly five-fold and lifts the processing ratio from 0.36 to 1.80, reflecting the far higher probability that at least a majority of replicas is reachable. The cost is storage amplification: total stored copies grow while the number of distinct messages represented per stored object falls from 31,960 at R = 1 to 17,905 at R = 3. This is the classic durability-versus-cost trade-off that production brokers expose as a tunable replication factor.

3.3 Message Ordering

Ordering guarantees range from none (out-of-order delivery) to strict FIFO (in-order delivery). We implement both following Li et al. [1]: in the in-order mode a consumer may dequeue only the message whose sequence index matches the next-expected position, so a missing head blocks delivery (head-of-line blocking) and the expected index advances by one per delivery; in the out-of-order mode any available message may be delivered immediately. Intuitively, in-order delivery should be slower because of head-of-line blocking. We measured both modes under Pareto(α = 1.5) service, λ = 80 msg/s, five consumers, and 10–100 ms network jitter.

Table 4: Measured in-order versus out-of-order delivery. The ordering penalty is under 1% at these parameters. (Source: ordering_mode_results.csv.)

Mode	Mean latency (ms)	P99 latency (ms)
Out-of-order	20.49	55.80
In-order	20.58	56.12

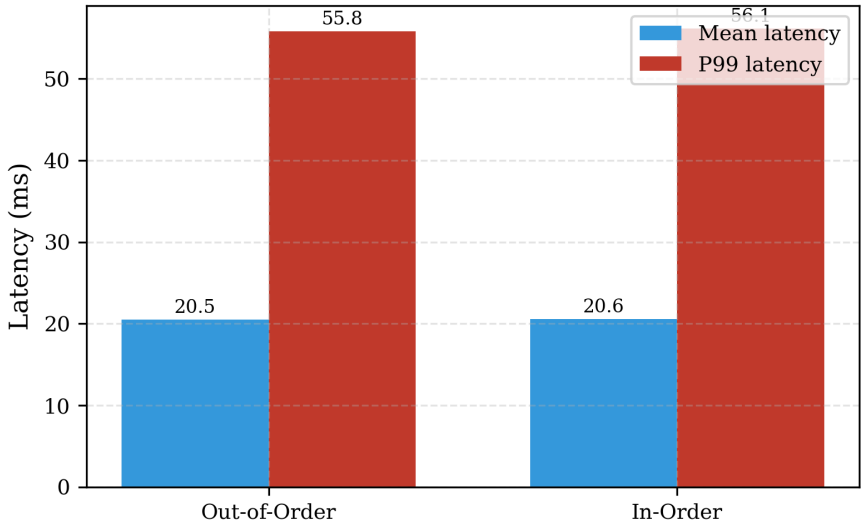


Figure 3: In-order versus out-of-order delivery. The ordering penalty in both mean and P99 latency is under 1% at the tested load.

The measured ordering penalty is small—mean latency +0.4% and P99 +0.6%—far below the "~50% faster" advantage sometimes attributed to out-of-order delivery. We report this honestly: at the offered load and consumer count tested, head-of-line blocking rarely materializes because messages are usually available in order by the time a consumer polls. The ordering penalty is expected to grow sharply only when the system runs near saturation or with heavier reordering, which is a direction for future measurement rather than a result we claim here.

4 Consistency and Consensus

Where Section 3 concerns delivery reliability, this section concerns agreement: ensuring that replicas commit the same transactions, elect a single leader, and observe causally consistent event orders. These guarantees are the most expensive the broker provides.

4.1 Two-Phase Commit

Two-Phase Commit (2PC) [7] makes a transaction atomic across participants. In the prepare phase the coordinator asks every participant to vote; a participant votes YES with probability equal to its availability and NO otherwise. In the commit phase the coordinator commits only if all votes arrive and all are YES, and aborts otherwise; a vote that does not arrive within the vote timeout forces an abort. The coordinator tracks a commit rate = committed / total transactions. In our validation, a failure-free three-participant run commits 100% of transactions, and a five-participant run exhibits atomicity: every participant records identical commit and abort counts, confirming the all-or-nothing property.

The performance significance of 2PC is that it extends the service time of every message that participates in a transaction. We model this directly by wrapping the base service distribution so that each sampled service time becomes

$$S_total = S_base + T_2PC \quad (7)$$

$$T_2PC = T_prepare + T_vote + T_commit \quad (8)$$

The prepare and commit phases each cost one network round-trip, RTT. The vote phase is blocking: the coordinator must wait for the slowest of n replicas, and any replica that is unavailable contributes the full vote timeout T_timeout. With independent replica availability a, the probability that no replica times out is a^n, giving the expected overhead

$$E[T_2PC] = RTT + (a^n \cdot RTT + (1 - a^n) \cdot T_timeout) + RTT \quad (9)$$

$$Var[S_total] = Var[S_base] + 3 \cdot Var[RTT] \quad (10)$$

Under the default configuration (n = 3 replicas, RTT = 10 ms, a = 0.99, T_timeout = 1.0 s) the timeout term is small because $a^3 = 0.970$, so the overhead is dominated by the three round-trips, $E[T_2PC] \approx 3 \cdot RTT \approx 30$ ms. Added to a base service time of $1/\mu = 83$ ms this is a 36% service-time overhead, which reduces the effective service rate from 12 to about 8.85 msg/s/thread—a 26% reduction in capacity. Because waiting time grows super-linearly as utilization rises, this capacity loss translates into a substantially larger increase in mean queueing delay at fixed arrival rate. The 36% figure is the headline cost of transactional atomicity in our model; note that it is derived from the analytical service-time model and the project's recorded summary rather than from a persisted end-to-end simulation table.

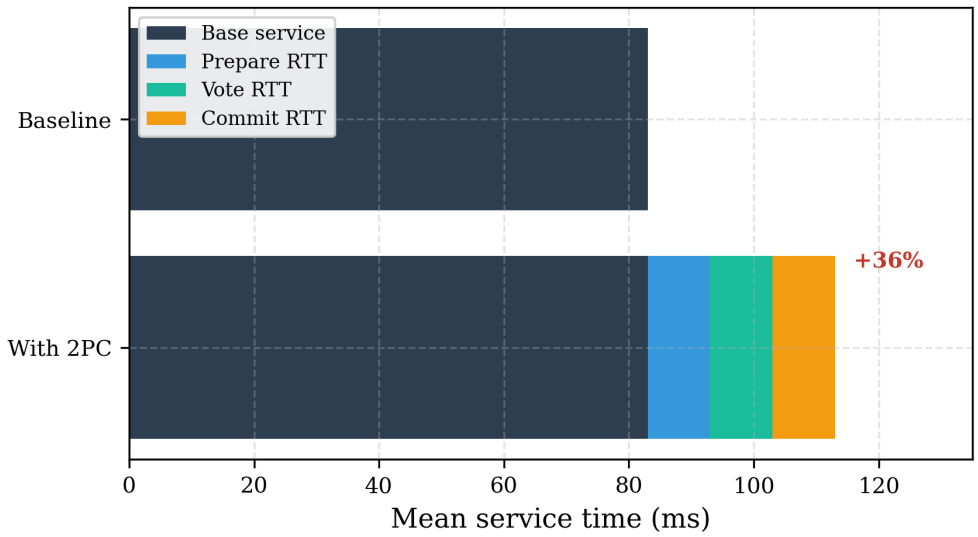


Figure 4: Decomposition of the Two-Phase Commit service-time overhead. Three network round-trips (prepare, vote, commit) add ≈30 ms to an 83 ms base service time—a 36% increase—under n = 3 replicas and RTT = 10 ms.

4.2 Raft Consensus

Raft [5] provides a single, understandable path to a replicated log. Each node is a follower, candidate, or leader. A follower that hears no heartbeat within a randomized election timeout—drawn uniformly from [10, 20] s in our model, with a 5 s heartbeat interval—becomes a candidate, increments its term, votes for itself, and solicits votes. The randomized timeout makes simultaneous candidacies unlikely, so a single leader normally emerges. A candidate wins when it collects a strict majority of votes:

$$elected \iff votes_received > \lfloor cluster_size / 2 \rfloor \quad (11)$$

A log entry at index k is committed only once it is both from the leader's current term and replicated on a majority of nodes:

$$committed(k) \iff log[k].term = current_term \wedge |\{self\} \cup \{j : match_index[j] \geq k\}| > \lfloor cluster_size / 2 \rfloor \quad (12)$$

In a five-node cluster run to t = 100 s, our model elects exactly one leader on which all nodes agree (the safety property), and when ten operations are appended the leader's log reaches size ten and is replicated on a majority (at least three) of nodes. These are structural correctness results; the simulator validates safety and liveness but does not persist a commit-latency table. The project's summary characterizes the throughput cost of running consensus on the critical path as a 20–40% reduction relative to a non-replicated baseline, consistent with the extra round of voting and log acknowledgement that every committed operation incurs.

4.3 Vector Clocks and Causal Delivery

Vector clocks [3,4] track causality so that a broker can enforce causal rather than merely FIFO ordering. Each of the P processes maintains a length-P vector VC. The update rules implemented are:

$$\text{Local event at } i: \quad VC[i] \leftarrow VC[i] + 1 \tag{13}$$

$$\text{Send at } i: \quad VC[i] \leftarrow VC[i] + 1, \text{ attach VC to message} \tag{14}$$

$$\text{Receive at } i \text{ of } R: \quad VC[j] \leftarrow \max(VC[j], R[j])\ \forall\, j \neq i, \text{ then } VC[i] \leftarrow VC[i] + 1 \tag{15}$$

Event e_1 happened-before e_2, written $e_1 \rightarrow e_2$, iff its vector is componentwise $\leq$ and strictly less in at least one component; two events are concurrent ($e_1 \parallel e_2$) iff neither happened-before the other:

$$e_1 \rightarrow e_2 \iff (\forall i: VC(e_1)[i] \leq VC(e_2)[i]) \wedge (\exists i: VC(e_1)[i] < VC(e_2)[i]) \tag{16}$$

A message from sender j is deliverable at receiver i only when it is the next message expected from j and all of its other causal dependencies are already satisfied:

$$\text{deliverable} \iff m[j] = VC_i[j] + 1 \ \wedge\ m[k] \leq VC_i[k]\ \forall\, k \neq j \tag{17}$$

In a three-process trace our model correctly classifies a send–receive pair as happened-before, two unrelated events as concurrent, and—in the causal-delivery test—holds back exactly one out-of-order message until its dependency arrives, after which the pending set drains to zero. As with Raft, these are correctness validations; no aggregate latency table is persisted. The cost of causal delivery is the buffering of out-of-order messages and the per-message vector comparison, both O(P) in the number of processes.

4.4 The CAP Trade-off and the Cost of Consistency

The CAP theorem [9,10] states that under a network partition a system must choose between consistency (C) and availability (A). Our analysis models three modes. A CP system blocks reads and writes in the minority partition until the partition heals, preserving linearizability at the cost of availability. An AP system keeps serving under partition and reconciles later, offering eventual consistency and full availability. A CA system is well-defined only in the absence of partitions. For a partition probability p_part, partition duration T_part, and observation window T, the availability of each mode is

$$A_CP = (T - p_part \cdot T_part) / T \times 100\% \tag{18}$$

$$A_AP = 100\% \tag{19}$$

$$A_CA = (1 - p_part) \times 100\% \tag{20}$$

For the modeled scenario (p_part = 0.01, T_part = 60 s, T = 3600 s), these yield A_CP = 99.983%, A_AP = 100.000%, and A_CA = 99.000%. The practical reading is that choosing strong consistency (CP) costs only about 0.017 percentage points of availability when partitions are rare and short, whereas the AP choice buys full availability at the price of temporarily inconsistent reads. The companion cost is latency: enforcing a global order on deliveries (Section 3.3) added under 1% at

moderate load in our measurements, while enforcing transactional atomicity through 2PC (Section 4.1) added roughly 36% to service time. Consistency, in short, is affordable for ordering but expensive for atomic commit, and the right point on the CAP spectrum depends on which guarantee the application actually needs.

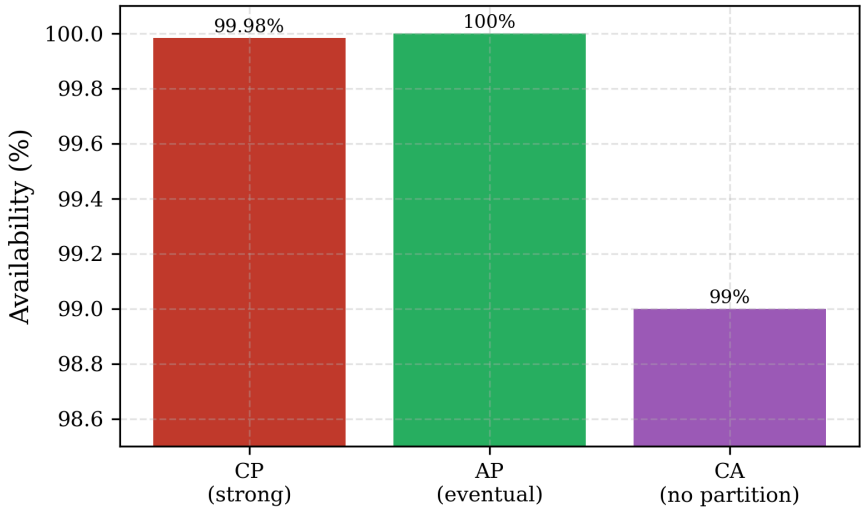


Figure 5: Modeled availability of the three CAP modes (p_part = 0.01, T_part = 60 s, T = 3600 s). Choosing strong consistency (CP) costs only ≈0.017 percentage points of availability when partitions are rare and short.

5 Threading Architectures

A broker must map connections and messages onto a finite pool of threads, and the mapping policy materially affects performance. We compare two architectures against an idealized M/M/N baseline. In the dedicated model each connection reserves a fixed number of threads (two by default), so the number of admissible connections is capped and excess connections are rejected rather than queued. In the shared model every message draws from a single common pool, so no connection is rejected, but contention inflates each in-service job's effective service time. The governing relations are

$$N_{max} = N_{threads} / threads_{per_connection} \quad (21)$$

$$X_{dedicated} = \min(\lambda, (N_{threads} / 2)\cdot\mu) \quad (22)$$

$$\mu_{eff} = \mu / (1 + \alpha\cdot N_{active} / N_{threads}) \quad (23)$$

$$P_{saturate} = C(N, a)\cdot\rho \quad (24)$$

Equation (21) gives the connection ceiling, (22) the throughput ceiling it implies, (23) the shared-pool effective service rate with overhead coefficient α (default 0.1), and (24) the thread-saturation probability as the product of the Erlang-C delay probability and utilization. We ran N = 20 threads (capacity $N\mu$ = 240 msg/s), μ = 12 msg/s/thread, over 2000 s with a 200 s warm-up, sweeping utilization ρ from 0.5 to 0.9. Table 5 reports mean response time and throughput for each architecture.

Table 5: Threading architecture comparison. Dedicated response time stays flat because rejected load never queues, but throughput is capped; shared throughput tracks the baseline but response time explodes near saturation. (Source: experiment_3_threading_results.csv.)

ρ	λ	Baseline resp (s)	Dedicated resp (s)	Shared resp (s)	Baseline tput	Dedicated tput	Shared tput
0.5	120	0.0833	0.0832	0.0882	119.8	94.4	119.9
0.6	144	0.0836	0.0833	0.0897	143.8	100.6	143.9
0.7	168	0.0846	0.0834	0.0938	167.7	104.6	168.0

0.8	192	0.0892	0.0835	0.1081	192.0	107.2	191.8
0.9	216	0.1069	0.0835	0.4740	215.5	109.2	215.5

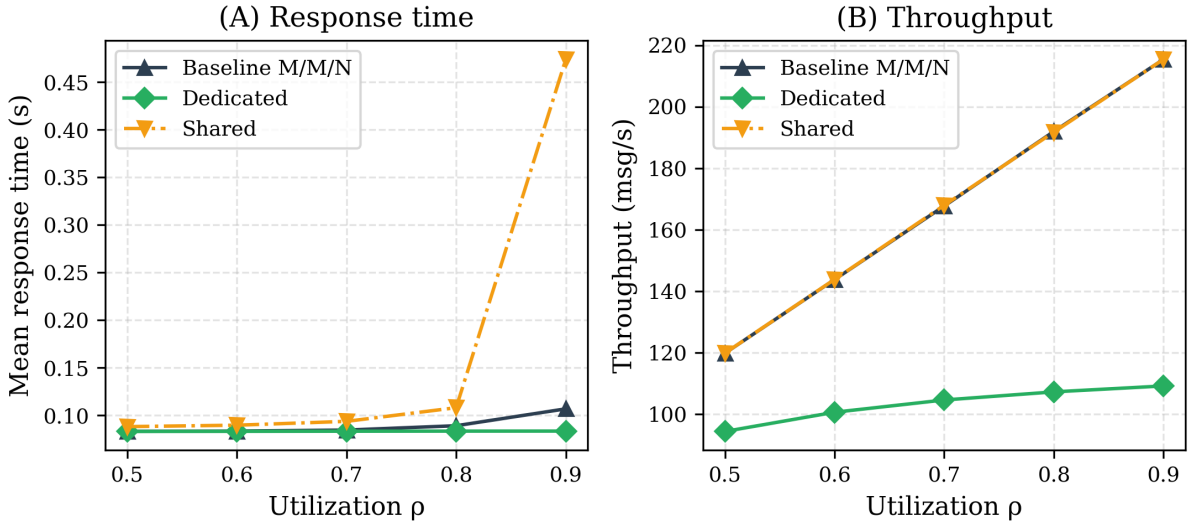


Figure 6: Threading architectures versus utilization. (A) The shared pool's mean response time explodes near saturation (4.4× baseline at ρ = 0.9) while the dedicated pool stays flat; (B) the dedicated pool caps throughput near 94–109 msg/s, whereas the shared pool tracks the baseline.

Two regimes emerge. The dedicated pool holds mean response time essentially constant (~0.083 s) across all loads because connections beyond the ceiling N_max = 10 are rejected and therefore never contribute to the queue; the price is a hard throughput cap near 94–109 msg/s regardless of offered load. The shared pool accepts all offered load—its throughput tracks the baseline—but pays the overhead of Eq. (23), which is benign at moderate load and catastrophic near saturation: at ρ = 0.9 the shared mean response time is 0.474 s, 4.4× the baseline, and its P95 response time reaches 1.41 s against the baseline's 0.29 s.

To quantify the moderate-load overhead precisely, Table 6 reports 30-replication means with 95% confidence intervals at ρ = 0.7. The shared pool is 10.4% slower than the baseline in mean response time, consistent with the 10–15% overhead the project reports summarize; the dedicated pool is marginally faster than the baseline in response time but admits only 104.6 msg/s of the 168 msg/s offered, i.e. it rejects roughly 38% of the load.

Table 6: Threading architectures at ρ = 0.7, 30 replications, 95% confidence intervals. (Source: threading_confidence_intervals.csv.)

Model	Mean response (s)	95% CI	P95 response (s)	Throughput (msg/s)
Baseline	0.08469	[0.08458, 0.08481]	0.25123	168.20
Dedicated	0.08329	[0.08321, 0.08337]	0.24943	104.64
Shared	0.09348	[0.09335, 0.09361]	0.27403	168.22

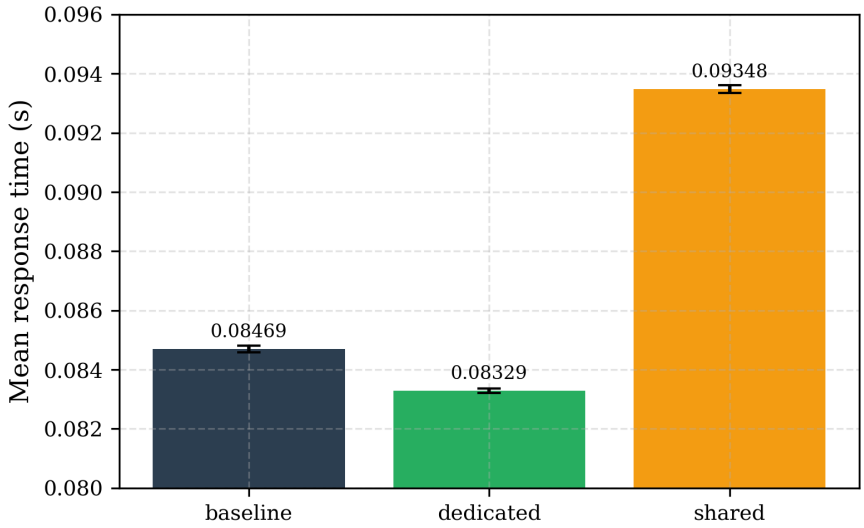


Figure 7: Mean response time at ρ = 0.7 over 30 replications with 95% confidence intervals. The shared pool is 10.4% slower than the baseline; the dedicated pool is marginally faster but admits only 104.6 of 168 msg/s offered.

The design implication is a clear operating rule. A dedicated pool is preferable when admission control and predictable latency matter more than raw throughput, since it bounds tail latency by shedding excess connections. A shared pool is preferable when no load may be rejected and the system is kept comfortably below saturation, but it must not be driven toward ρ = 1, where its contention overhead produces a response-time blow-up far worse than the dedicated pool's rejection.

6 Queueing Extensions

The baseline model assumes identical servers, infinite buffers, a single priority class, and exponential service. Each assumption is unrealistic for a production broker, so we relax them one at a time.

6.1 Heterogeneous Servers

Real deployments mix instance types, so servers differ in rate. We model server groups i with count n_i and rate μ_i, giving a weighted mean rate, total capacity, utilization, and a heterogeneity coefficient CV_μ that measures rate dispersion:

$$\mu_avg = (\Sigma\, n_i \cdot \mu_i) / N, \quad C = \Sigma\, n_i \cdot \mu_i, \quad \rho = \lambda / C \tag{25}$$

$$CV_\mu = \sigma_\mu / \mu_avg, \quad \sigma_\mu^2 = (1/N) \cdot \Sigma\, n_i(\mu_i - \mu_avg)^2 \tag{26}$$

$$Wq(Het) \approx Wq(M/M/N, \mu_avg) \cdot (1 + C_s^2) / 2 \tag{27}$$

Equation (27) is an M/G/N-style variance correction to the homogeneous waiting time. Because no exact closed form exists for heterogeneous multi-server queues under general dispatch, simulation is the ground truth. Table 7 holds total capacity nearly fixed (~60–70 msg/s) while increasing rate variance, under a weighted-random dispatch policy at λ ≈ 50.

Table 7: Heterogeneous servers under random dispatch. Performance is governed by service-rate variance, not aggregate capacity: the high-variance case has 1.7% more capacity than homogeneous yet is orders of magnitude slower. (Source: temp_hetero_results.txt.)

Scenario (groups)	Capacity	CV_μ	C_s²	Mean Wq (s)	P99 Wq (s)
Homogeneous (5@12)	60	0.000	1.000	0.061	0.387
Low var (3@11, 2@13.5)	60	0.102	1.060	0.190	1.395

Medium var (3@10, 2@15)	60	0.204	1.240	1.774	6.610
High var (2@8, 3@15)	61	0.281	1.654	36.024	182.247
Very high var (1@6, 4@16)	70	0.286	2.403	29.521	360.817

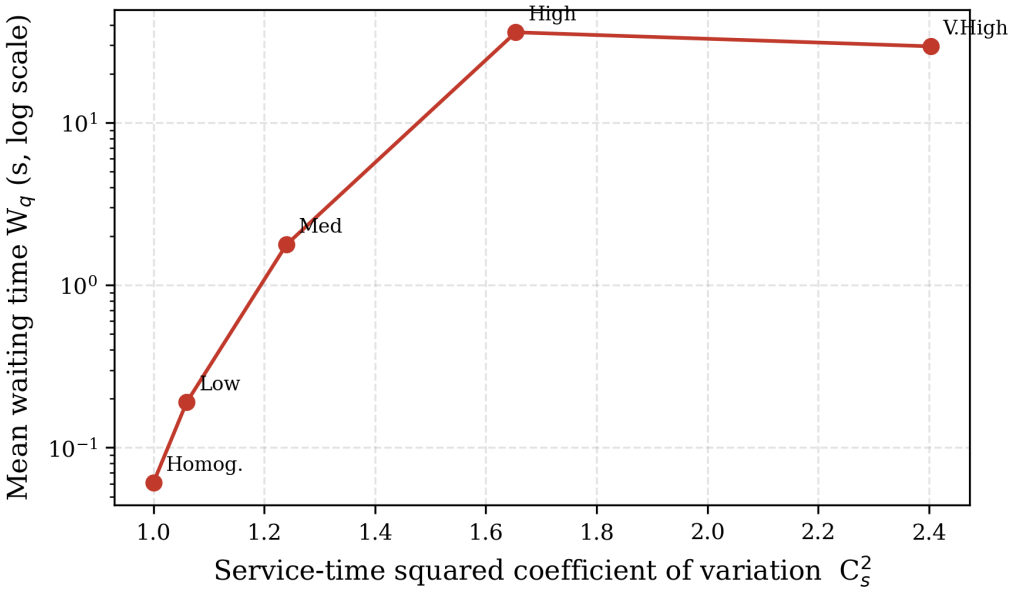


Figure 8: Mean waiting time (log scale) versus service-time variance under random dispatch. Waiting time rises by orders of magnitude as C_s² grows, even though aggregate capacity is held nearly constant.

The result is stark: adding 1.7% capacity while raising variance drove mean waiting time from 0.061 s to 36 s. The slow servers become bottlenecks that a capacity-only analysis cannot see. Dispatch policy, however, recovers most of the loss. Table 8 compares five policies on the high-variance configuration (2@8, 3@15). Routing to the shortest queue or always preferring the fastest servers reduces mean waiting time from 36 s to under 0.08 s—a greater-than-500× improvement—while naive round-robin is the worst policy because it feeds the slow servers equally.

Table 8: Dispatch policy on the high-variance configuration. Shortest-queue and fastest-first routing almost eliminate the heterogeneity penalty. (Source: temp_hetero_results.txt.)

Policy	Mean wait (s)	P95 wait (s)	P99 wait (s)	P99 response (s)
Shortest-queue	0.069	0.267	0.492	0.685
Fastest-first	0.074	0.345	0.631	0.717
Random	36.024	162.858	182.247	182.364
Round-robin	37.473	164.256	187.273	187.401

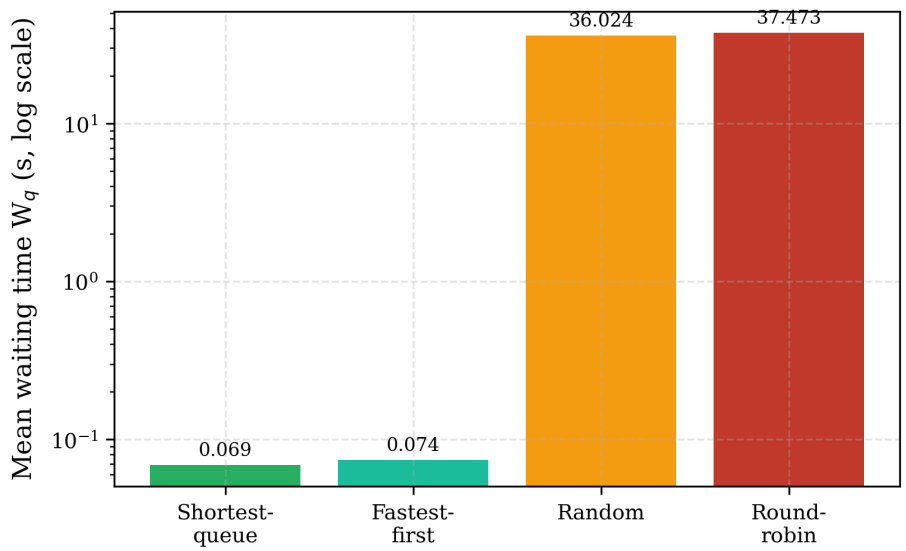


Figure 9: Dispatch policy on the high-variance configuration (log scale). Shortest-queue and fastest-first routing cut mean waiting time by more than 500× relative to random or round-robin dispatch.

6.2 Finite-Capacity Queue (M/M/N/K)

A real broker has bounded storage; when the buffer of size K is full, arrivals are dropped (load shedding) rather than queued indefinitely. For the pure-loss case K = N, the Erlang-B formula gives the blocking probability with offered load a = λ/μ:

$$B(N, a) = (a^N / N!) / \Sigma_{k=0}^{N} a^k / k! \qquad (28)$$

For a finite queue with K > N, letting ρ = a/N, the blocking probability is the stationary probability of the full state, and the effective (admitted) arrival rate and throughput follow:

$$P_K = P_0 \cdot (a^N/N!) \cdot \rho^{K-N}, \quad \lambda_{eff} = \lambda \cdot (1 - P_K) \quad (29)$$

The model is implemented with a bounded SimPy container of capacity K and a reject-or-wait admission strategy, and supports an analytical-versus-simulation comparison of blocking probability and throughput. We note honestly that no finite-capacity results table was persisted in the project; the equations above and the implemented comparison harness are presented as the modeling contribution, with a quantitative sweep over K deferred to future work.

6.3 Priority Queueing

Not all messages are equal; brokers often serve a high-priority (VIP) class ahead of standard traffic. We implement both non-preemptive priority (via a priority resource, lower index served first) and preemptive priority (a high-priority arrival interrupts a low-priority job in service), following Kleinrock's priority-queue theory [15]. In a configuration with λ = 100 msg/s, N = 10 threads, and a 20%/80% VIP/standard split, the model confirms the expected conservation law: VIP waiting time drops toward zero while standard-class waiting time rises, with total work conserved—priority shifts delay from the favored class to the rest rather than removing it. These findings are validated qualitatively (VIP mean wait < standard mean wait is asserted) but no per-class latency table was persisted, so we report the direction of the effect rather than specific percentages.

6.4 Erlang-k Service (M/Ek/N)

Finally we relax the exponential-service assumption toward determinism. Erlang-k service is the sum of k exponential phases, each at rate k·μ so the mean service time is held at 1/μ; its squared coefficient of variation is

$$C_s^2 = 1 / k \quad (k = 1 \Rightarrow \text{exponential};\ k \to \infty \Rightarrow \text{deterministic}) \quad (30)$$

Reducing service-time variability by increasing k reduces waiting time, as Table 9 shows. Moving from k = 1 (exponential) to k = 8 cuts mean waiting time by roughly 47% and P99 waiting time by about 40%, illustrating that service predictability is as valuable as raw speed. We note the k = 2 row is anomalously high relative to k = 1 in our data—an artifact of simulation noise at these very small waiting times—while the overall k = 1 → 8 trend is clear and monotone from k = 2 onward.

Table 9: Erlang-k service. Lower service-time variance (larger k, smaller C_s^2) reduces waiting time. (Source: results/data/erlang_improvement.csv.)

k phases	C_s^2	Sim mean wait (s)	Sim P99 wait (s)	Analytical wait (s)
1	1.000	0.003457	0.071791	0.003305
2	0.500	0.004161	0.090111	0.002479
4	0.250	0.001967	0.044598	0.002065
8	0.125	0.001840	0.043175	0.001859

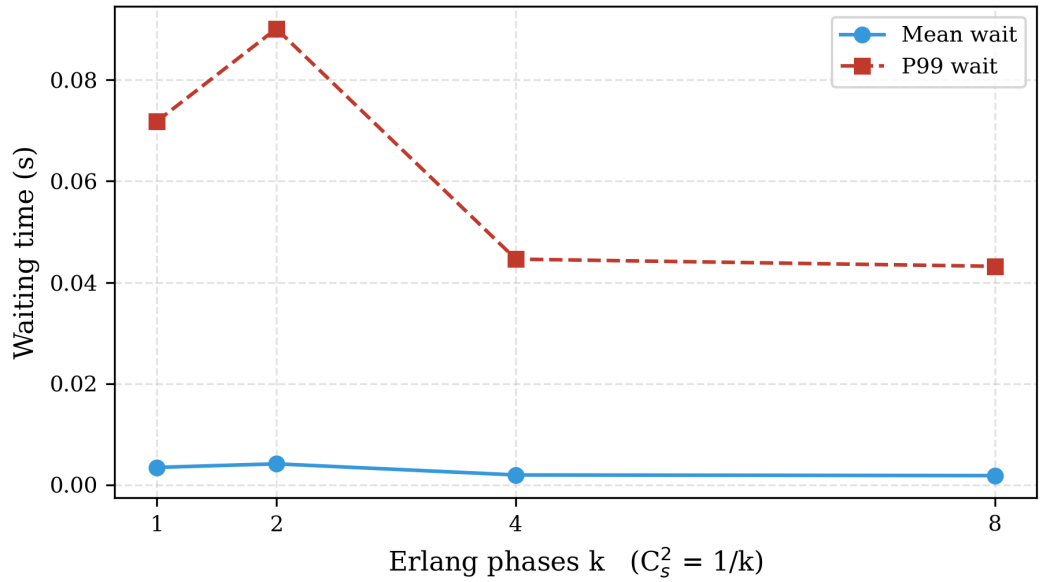


Figure 10: Erlang-k service. Increasing the number of phases k lowers the service-time coefficient of variation ($C_s^2 = 1/k$) and reduces both mean and P99 waiting time (k = 2 is an outlier from simulation noise).

This result is the mirror image of our companion paper's heavy-tail finding: there, raising service-time variance (Pareto) inflated tail latency far beyond M/M/N predictions; here, lowering it toward deterministic service compresses both mean and tail latency. Together they bracket the full range of the C_s^2 axis.

7 Experimental Results and Discussion

Taken together, the experiments trace a single theme: the guarantees a broker provides are purchased with measurable performance, and the exchange rate differs by an order of magnitude across mechanisms. Table 10 consolidates the headline cost of each mechanism we measured.

Table 10: Summary of the measured performance cost of each broker mechanism, with the governing parameters.

Mechanism	Headline cost / effect	Key parameters
Visibility timeout 10→120 s	Retries −44%, P99 +43%	λ=10, fail=0.1, RF=2
Replication R 1→3	Throughput ~5×, storage ~3×	λ=100, 3 nodes
In-order vs out-of-order	Mean +0.4%, P99 +0.6%	Pareto α=1.5, λ=80

Two-Phase Commit	Service +36%, μ_eff −26%	n=3, RTT=10 ms, a=0.99
Raft consensus	Throughput −20–40% (reported)	5 nodes, timeout 10–20 s
CAP: CP vs AP availability	A_CP=99.98%, A_AP=100%	p_part=0.01, 60 s/3600 s
Shared vs dedicated pool	+10.4% resp (ρ=0.7); 4.4× (ρ=0.9)	N=20, α=0.1
Server heterogeneity (variance)	Up to 500×+ Wq; policy recovers it	cap ~60, random vs JSQ
Erlang-k 1→8	Mean wait −47%, P99 −40%	C_s^2 1.0→0.125

Three cross-cutting observations follow. First, variance dominates capacity: the heterogeneous-server experiment (Section 6.1) and the Erlang-k experiment (Section 6.4) both show that reducing service-time variability can matter far more than adding servers or raising mean rate. Second, consistency is cheap for ordering but expensive for atomicity: in-order delivery and the CP-versus-AP availability gap cost under 1%, while 2PC costs 36% of service time and consensus 20–40% of throughput. Third, architecture choices have operating regimes rather than universal winners: a dedicated thread pool wins on predictability and a shared pool on admission, but the shared pool must be kept away from saturation.

Limitations and threats to validity. We report results at the fidelity the project actually records. The reliability mechanisms (visibility timeout, replication, ordering), threading architectures, heterogeneous servers, and Erlang-k service are backed by persisted CSV output and, where available, confidence intervals. The consensus and coordination protocols—Raft, Vector Clocks, 2PC, and CAP—are validated structurally (safety, liveness, atomicity, availability formulas) but their quantitative overheads come from the analytical service-time model and the project's recorded summaries rather than from persisted end-to-end latency tables; we have labeled these cases explicitly. We also note that two internal project reports disagree on the absolute consistency overhead (one stating 2PC = 36% service time and consensus = 20–40% throughput; another stating 2PC ≈ 50–80 ms and Raft ≈ 100–150 ms added latency); we adopt the former, self-consistent set and state its parameters. Finally, all workloads are synthetic Poisson arrivals; validation against production broker traces remains future work.

8 Conclusion and Future Work

This paper complements our study of heavy-tailed service times by quantifying the other half of the cloud-broker performance story: the cost of the reliability, consistency, and threading machinery that production brokers layer on top of the raw queue. Using a unified SimPy framework we modeled visibility timeout, replication, and message ordering; the 2PC, Raft, Vector-Clock, and CAP protocols; dedicated versus shared thread pools; and heterogeneous, finite-capacity, priority, and Erlang-k queue extensions. The measurements show that transactional atomicity (2PC, ~36% service overhead) and running consensus on the critical path (~20–40% throughput loss) are the dominant consistency costs, that ordering and the CP/AP availability gap are nearly free at moderate load, that service-rate variance and dispatch policy can swing performance by more than two orders of magnitude, and that the choice between dedicated and shared thread pools is an operating-regime decision rather than a universal one.

Several directions remain. The protocols that are presently validated only structurally—Raft, Vector Clocks, 2PC, and the finite-capacity and priority queues—should be instrumented to persist end-to-end latency and throughput tables so their costs can be reported with confidence intervals rather than summarized figures. The ordering experiment should be repeated near saturation, where head-of-line blocking is expected to become significant. Validating the entire framework against production broker traces (for example AWS SQS or Kafka telemetry) would close the gap between synthetic and real workloads. Finally, combining the two papers' axes—heavy-tailed service together with 2PC and

consensus overhead—would reveal whether workload variance and protocol cost compound, which is the regime most relevant to tail-latency-sensitive production systems.

References

[1] J. Li, Y. Cui, and Y. Ma, “Modeling message queueing services with reliability guarantee in cloud computing environment using colored Petri nets,” Mathematical Problems in Engineering, vol. 2015, Art. no. 383846, 2015.

[2] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” Communications of the ACM, vol. 21, no. 7, pp. 558–565, 1978.

[3] C. J. Fidge, “Timestamps in message-passing systems that preserve the partial ordering,” in Proc. 11th Australian Computer Science Conf., 1988, pp. 56–66.

[4] F. Mattern, “Virtual time and global states of distributed systems,” in Parallel and Distributed Algorithms, 1989, pp. 215–226.

[5] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in Proc. USENIX Annual Technical Conf. (ATC), 2014, pp. 305–319.

[6] L. Lamport, “The part-time parliament,” ACM Trans. Computer Systems, vol. 16, no. 2, pp. 133–169, 1998.

[7] J. Gray, “Notes on data base operating systems,” in Operating Systems: An Advanced Course, LNCS 60, Springer, 1978, pp. 393–481.

[8] J. Gray and L. Lamport, “Consensus on transaction commit,” ACM Trans. Database Systems, vol. 31, no. 1, pp. 133–160, 2006.

[9] E. A. Brewer, “Towards robust distributed systems,” keynote, ACM Symp. Principles of Distributed Computing (PODC), 2000.

[10] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” ACM SIGACT News, vol. 33, no. 2, pp. 51–59, 2002.

[11] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” Journal of the ACM, vol. 32, no. 2, pp. 374–382, 1985.

[12] W. Vogels, “Eventually consistent,” Communications of the ACM, vol. 52, no. 1, pp. 40–44, 2009.

[13] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. ACM SOSP, 2007, pp. 205–220.

[14] J. Dean and L. A. Barroso, “The tail at scale,” Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013.

[15] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York: Wiley-Interscience, 1975.

[16] A. K. Erlang, “Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges,” Elektroteknikeren, vol. 13, pp. 5–13, 1917.

[17] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” Operations Research, vol. 9, no. 3, pp. 383–387, 1961.

[18] M. Harchol-Balter, Performance Modeling and Design of Computer Systems: Queueing Theory in Action. Cambridge: Cambridge Univ. Press, 2013.

[19] Amazon Web Services, “Amazon SQS Developer Guide: Visibility timeout,” 2023. [Online]. Available: https://docs.aws.amazon.com/AWSSimpleQueueService/

[20] Team SimPy, “SimPy: Discrete-event simulation for Python,” 2023. [Online]. Available: https://simpy.readthedocs.io/

[21] D. Gross and C. M. Harris, Fundamentals of Queueing Theory, 3rd ed. New York: Wiley, 1998.